Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Tarea N°5

Clustering

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 1 de julio de 2024
Fecha de entrega: 30 de junio de 2024
Santiago de Chile

Introducción

En el presente informe detalla el desarrollo de la tarea N°5 del curso de Inteligencia Computacional. El desarrollo de la tarea se hizo acorde a las indicaciones del documento *Tarea 5 EL4106 2024 v04.pdf*.
En el documento mencionado anteriormente, se tiene por objetivo la implementación y análisis del desempeño de diversos algoritmos de clustering. Se utilizara el conjunto de datos *Mice Protein Expression Data Set* de UC Irvine Machine Learning Repository, el cual fue entregado por el cuerpo docente en el archivo *Data_Cortex_Nuclear.xls*. El conjunto inicial posee 80 características, 8 clases y 1080 muestras en total.
Por ultimo, los objetivos de la tarea, y por tanto lo que se ha realizado, consisten en:

Manejo del Dataset Esta sección consiste en: La lectura del Data Set y la modificación del mismo.

Modificación de *bench_k_means* Esta sección consiste en: La modificación de la función *bench_k_means* agregando una columna extra correspondiente al numero de clusters.

Implementación de *bench_clustering2* Esta sección consiste en: La creación de una nueva función *bench_clustering2* a partir de la función *bench_k_means*. Se elimina la columna *inertia*, pues los algoritmos *DBSCAN* y *Hierarchiral Agglomerative* no la poseen. Además, la nueva función maneja casos borde del algoritmo *DBSCAN*.

Entrenamiento de los algoritmos de clustering Esta sección consiste en: El entrenamiento de los algoritmos se realiza mediante el uso de las funciones definidas previamente. Los algoritmos de clustering fueron implementados mediante *scikit-learn*.
Los algoritmos implementados son:

K-Means Con inicialización *random* y *K-Means++* y 8 clusters.

DBSCAN Con épsilon por defecto, 0.2 y 0.7. Considerando las muestras o sin considerar las muestras ruido.

Hierarchical Agglomerative Con 8 clusters.

Posteriormente se repite el entrenamiento posterior al uso de un *PCA* sobre los datos para reducirlos a 2 dimensiones.

Análisis de los resultados Esta sección consiste en:

Comparación de algoritmos En base a las métricas *completeness*, *homogenity*, *v-measure* y *silhoutte*. Indicando:

1. Cual es el mejor algoritmo según cada métrica previamente mencionada.
2. Cual es el peor algoritmo según cada métrica previamente mencionada.

Se deben considerar tanto los algoritmos originales como los algoritmos en los que se aplico *PCA*.

DBSCAN Se analizo lo siguiente:

1. Casos sin métricas.
2. Numero de clusters y su efecto sobre las métricas.
3. Efecto de considerar o no las muestras ruido y su efecto sobre las métricas.

PCA Efectos de utilizar *PCA* sobre las métricas.

Índice de Contenidos

1. Desarrollo **1**
1.1. Manejo del Dataset . . . 1
1.2. Modificación *bench_k_means* . . . 1
1.3. Implementación de *bench_clustering2* . . . 3
1.4. Entrenamiento de los algoritmos de clustering . . . 5
1.4.1. K-Means . . . 6
1.4.2. K-Means++ . . . 6
1.4.3. DBSCAN . . . 6
1.4.4. DBSCAN $\epsilon = 0.7$ . . . 6
1.4.5. DBSCAN $\epsilon = 0.2$ . . . 7
1.4.6. DBSCAN con ruido . . . 7
1.4.7. DBSCAN $\epsilon = 0.7$ con ruido . . . 7
1.4.8. DBSCAN $\epsilon = 0.2$ con ruido . . . 7
1.4.9. Hierarchical Agglomerative . . . 8
1.4.10. Repetición con PCA . . . 8
1.5. Análisis de los resultados . . . 8
1.5.1. Comparación de algoritmos . . . 9
1.5.2. Algoritmo DBSCAN . . . 10
1.5.2.1 Casos sin métricas . . . 10
1.5.2.2 Numero de clusters . . . 10
1.5.2.3 Efecto de considerar las muestras ruido . . . 11
1.5.3. Efecto de utilizar PCA . . . 11

2. Conclusiones **12**
2.1. Aprendizajes . . . 12
2.2. Dificultades . . . 12
2.3. Evaluación de resultados . . . 12

Anexo A. Código **13**

Índice de Tablas

1. Algoritmos. . . . 5
2. Resultados de `KMeans_1`. . . . 6
3. Resultados de `KMeans_2`. . . . 6
4. Resultados de `DBSCAN_1` sin muestras ruido. . . . 6
5. Resultados de `DBSCAN_2` sin muestras ruido. . . . 6
6. Resultados de `DBSCAN_3` sin muestras ruido. . . . 7
7. Resultados de `DBSCAN_1` con muestras ruido. . . . 7

8. Resultados de `DBSCAN_2` con muestras ruido. 7
9. Resultados de `DBSCAN_3` con muestras ruido. 7
10. Resultados de `Agglomerative_1`. 8
11. Resultados de `KMeans_1`. 8
12. Algoritmos con las mejores métricas. 9
13. Algoritmos con las peores métricas. 9
14. Casos sin métricas 10
15. Numero de clusters de los algoritmos DBSCAN. 11

Índice de Códigos

1. Carga y modificación del Data Set. 1
2. Modificación de *bench_k_means* 1
3. Implementación de *bench_clustering2* 3
4. Transformación de DataFrame en arreglos. 5
5. Inicialización de algoritmos. 5
6. *PCA* y Algoritmos con *PCA*. 5
7. Entrenamiento con *PCA*. 8
A.1. Código. 13

1. Desarrollo

1.1. Manejo del Dataset

En la presente sección se realizo la carga del Dataset al entorno de Google Colaboratory. Para la lectura del Dataset se utilizo la librería *pandas*, lo que genero un DataFrame.
Además, el Dataset contenido en *Data_Cortex_Nuclear.xls* posee una columna llamada *BCL2_N*, la cual tiene muchos valores *NaN*, por lo cual fue eliminada. En esta misma linea, existen varias filas que contienen valores *NaN*, dichas filas también fueron eliminadas. Originalmente, el DataFrame poseía 82 columnas y 1080 filas. Posterior a la modificación, el DataFrame posee 81 columnas y 612 filas.
Las 81 columnas están compuestas por 79 columnas de características, 1 columna de id y 1 columna de clase.
El código implementado para la carga y el manejo del Dataset se presenta en 1.

Código 1: Carga y modificación del Data Set.

```
DataSet = 'Data_Cortex_Nuclear.xls'
DataFrame = pd.read_excel(DataSet)
DataFrame = DataFrame.drop('BCL2_N', axis=1)
DataFrame = DataFrame.dropna()
```

1.2. Modificación *bench_k_means*

En la presente sección se realizo a la modificación de la función *bench_k_means*. Dicha modificación consiste en el agregado de una columna extra correspondiente al numero de clusters y el agregado de un encabezado para los resultados de la función.
El código implementado se presenta en 2.

Código 2: Modificación de *bench_k_means*

```
def bench_k_means(kmeans, name, data, labels):
    t0 = time()
    estimator = make_pipeline(kmeans).fit(data)
    fit_time = time() - t0
    results = [name, fit_time, estimator[-1].inertia_]
    clustering_metrics = [
        metrics.homogeneity_score,
        metrics.completeness_score,
        metrics.v_measure_score,
        metrics.adjusted_rand_score,
        metrics.adjusted_mutual_info_score,
    ]
    results += [m(labels, estimator[-1].labels_) for m in clustering_metrics]
    results += [
        metrics.silhouette_score(
```

```
16          data,
17          estimator[-1].labels_,
18          metric="euclidean",
19          sample_size=300,
20      )
21  ]
22  results.append(kmeans.n_clusters)
23  header = ["init", "time", "inertia", "homo", "compl", "v-meas", "ARI", "AMI", "silhouette",
    ↪ "n_clusters"]
24  formatter_header = (
25      "{:8s}\t{:4s}\t{:7s}\t{:4s}\t{:5s}\t{:6s}\t{:3s}\t{:3s}\t{:10s}\t{:4s}"
26  )
27  formatter_result = (
28      "{:8s}\t{:.3f}s\t{:.0f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:9.0f}"
29  )
```

Lo agregado y modificado se encuentra en:

Linea 22 Se agrego `results.append(kmeans.n_clusters)`, lo cual agrega la cantidad de clusters a la lista de resultados.

Linea 23 Se agrego `header = ["init", "time", "inertia", "homo", "compl", "v-meas", "ARI", "AMI ↪ ", "silhouette", "n_clusters"]`, lo cual agrega genera una lista con los encabezados.

Lineas entre 24 **y** 26 Se agrego `formatter_header = ("{:8s}t{:4s}t{:7s}t{:4s}t{:5s}t{:6s}t ↪ {:3s}t{:3s}t{:10s}t{:4s}")`, lo cual formatea el encabezado.

Lineas entre 27 **y** 29 Se agrego `t{:9.0f}` al final del `formater_result` original, lo cual formatea los resultados, incluyendo la cantidad clusters.

Es importante mencionar que las métricas que entrega la función son:

Time Tiempo de entrenamiento del algoritmo.

Inertia Mide la suma de las distancias cuadradas entre cada punto de datos y el centroide del cluster al que pertenece.

Homogeneity Score Mide la homogeneidad de los clusters, es decir, si cada cluster contiene solo miembros de una sola clase.

Completeness Score Evalúa cuan completa es la asignación de miembros de cada clase en un solo cluster.

V Measure Score Es la media armónica entre las dos metricas anteriores.

Adjusted Rand Score Ajusta el índice Rand para tener en cuenta la probabilidad de coincidencias aleatorias entre dos clusters.

Adjusted Mutual Info Score Calculo de la cantidad de información mutua ajustada entre los clusters predichos y los verdaderos.

Silhoutte Score Evalúa la cohesión y separación de los clusters, entre si mismo y entre otros.

N Clusters Indica la cantidad de clusters generados por el algoritmo.

1.3. Implementación de *bench_clustering2*

En la presente sección se implemento la función *bench_clustering2*, la cual toma como base *bench_k_means*.
Esta implementación fue utilizada sobre los algoritmos *DBSCAN* y *Hierarchical Agglomerative*, por lo se elimino *inertia* de los resultados, pues dichos algoritmos no poseen esa métrica. Además, la función implementada maneja casos borde del algoritmo *DBSCAN*:

Muestras ruido Se agrego una variable booleana que determina si las muestras ruido se consideran como un cluster adicional o no.

Cantidad de clusters ≤ 1 Debido a que no se pueden calcular las métricas, se retorna una lista compuesta por 6 valores *NaN* y el numero de clusters.

El código implementado se presenta en 3.

Código 3: Implementación de *bench_clustering2*

```
def bench_clustering2(method, name, data, labels, calculate_all):
    t0 = time()
    estimator = make_pipeline(method).fit(data)
    fit_time = time() - t0
    results = [name, fit_time]
    clustering_metrics = [
        metrics.homogeneity_score,
        metrics.completeness_score,
        metrics.v_measure_score,
        metrics.adjusted_rand_score,
        metrics.adjusted_mutual_info_score,
    ]
    if isinstance(method, sklearn.cluster.DBSCAN):
      n_clusters = len(set(estimator[-1].labels_)) - (1 if -1 in estimator[-1].labels_ else 0)
      if calculate_all and -1 in estimator[-1].labels_:
        predicted_labels = np.where(estimator[-1].labels_ == -1, n_clusters, estimator[-1].
     ↪ labels_)
        n_clusters += 1
    elif isinstance(method, sklearn.cluster.AgglomerativeClustering):
      n_clusters = method.n_clusters
    if n_clusters > 1:
      if not calculate_all and isinstance(method, sklearn.cluster.DBSCAN):
```

```
22        mask = estimator[-1].labels_ != -1
23      else:
24        mask = np.ones(len(estimator[-1].labels_), dtype=bool)
25      results += [m(labels[mask], estimator[-1].labels_[mask]) for m in clustering_metrics]
26      results += [
27          metrics.silhouette_score(
28              data[mask],
29              estimator[-1].labels_[mask],
30              metric="euclidean",
31              sample_size=300,
32          )
33      ]
34    else:
35      results += [np.nan] * 6
36    results.append(n_clusters)
37    header = ["init", "time", "homo", "compl", "v-meas", "ARI", "AMI", "silhouette", "
       ↪ n_clusters"]
38    formatter_header = (
39        "{:8s}\t{:4s}\t{:4s}\t{:5s}\t{:6s}\t{:3s}\t{:3s}\t{:10s}\t{:4s}"
40    )
41    formatter_result = (
42        "{:8s}\t{:.3f}s\t{:.0f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:10.0f}"
43    )
```

Lo agregado se encuentra en:

Linea 5 Se elimino la métrica *inertia*.

Lineas entre 13 **y** 19 Se implemento un método para calcular la cantidad de clusters para cada algoritmo.

Linea 20 Define los cálculos a realizar si existe más de un cluster.

Lineas entre 21 **y** 24 Se implemento un método para generar una mascara, la cual varia según el valor de *calculate_all*. Cuando dicho valor es Falso, no se consideran las muestras ruido, en caso contrario si se consideran.

Lineas entre 25 **y** 33 Se implemento un método que calcula las métricas según la mascara previamente generada.

Lineas entre 34 **y** 35 Se implemento un método que al no encontrar más de un cluster, los resultados de las métricas se definen como *NaN*.

Linea 36 Se agrego `results.append(n_clusters)`, lo cual agrega la cantidad de clusters a la lista de resultados.

Las métricas contenidas en los resultados son las mismas que las de 1.2, exceptuando por *inertia*, la cual fue eliminada.

1.4. Entrenamiento de los algoritmos de clustering

En la presente sección se realizo el entrenamiento de los distintos algoritmos de clustering. Para lo anterior fue necesario transformar el DataFrame en arreglos de *numpy*. Específicamente, un arreglo con los datos y un arreglo con las clases.
El código implementado para realizar lo anterior se presenta en 4.

Código 4: Transformación de DataFrame en arreglos.

```
data = DataFrame.drop('class', axis=1).select_dtypes(include=['float64', 'int64']).values
labels = DataFrame['class'].values
```

Luego, se inicializaron los algoritmos:

Tabla 1: Algoritmos.

Algoritmo	Parámetros	Cantidad de clusters
K-Means	Inicialización *random*	8
K-Means	Inicialización *K-Means++*	8
DBSCAN	Con ϵ por defecto	N/A
DBSCAN	Con $\epsilon = 0.7$	N/A
DBSCAN	Con $\epsilon = 0.2$	N/A
Hierarchical Agglomerative	N/D	8

El código implementado se presenta en 5.

Código 5: Inicialización de algoritmos.

```
KMeans_1 = KMeans(init='random', n_clusters=8, n_init=10)
Kmeans_2 = KMeans(init='k-means++', n_clusters=8 , n_init=10)
DBSCAN_1 = DBSCAN()
DBSCAN_2 = DBSCAN(eps=0.7)
DBSCAN_3 = DBSCAN(eps=0.2)
Agglomerative_1 = AgglomerativeClustering(n_clusters=8)
```

Además, se realizo un *PCA* sobre el arreglo de los datos, en el cual se disminuyo el arreglo a 2 dimensiones y se inicializaron los algoritmos de 1 con el arreglo reducido.
El código implementado se presenta en 6.

Código 6: *PCA* y Algoritmos con *PCA*.

```
reduced_data = PCA(n_components=2).fit_transform(data)
KMeans_1_PCA = KMeans_1.fit(reduced_data)
Kmeans_2_PCA = Kmeans_2.fit(reduced_data)
DBSCAN_1_PCA = DBSCAN_1.fit(reduced_data)
DBSCAN_2_PCA = DBSCAN_2.fit(reduced_data)
DBSCAN_3_PCA = DBSCAN_3.fit(reduced_data)
```

```
Agglomerative_1_PCA = Agglomerative_1.fit(reduced_data)
```

1.4.1. K-Means

En la presente sección se entreno al algoritmo `KMeans_1` mediante el uso de la función implementada en 1.2, por medio de `bench_k_means(KMeans_1, "KMeans_1", data, labels)`. Los resultados obtenidos se presentan en 2.

Tabla 2: Resultados de `KMeans_1`.

init	time	inertia	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
KMeans_1	0.051s	1123	0.290	0.291	0.290	0.144	0.276	0.199	8

1.4.2. K-Means++

En la presente sección se entreno al algoritmo `KMeans_2` mediante el uso de la función implementada en 1.2, por medio de `bench_k_means(KMeans_2, "KMeans_2", data, labels)`. Los resultados obtenidos se presentan en 3.

Tabla 3: Resultados de `KMeans_2`.

init	time	inertia	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
KMeans_2	0.557s	1123	0.300	0.300	0.300	0.147	0.286	0.215	8

1.4.3. DBSCAN

En la rpesente sección se entreno al algoritmo `DBSCAN_1` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_1, "DBSCAN_1", data, labels, False)`.
Los resultados obtenidos se presentan en 4.

Tabla 4: Resultados de `DBSCAN_1` sin muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_1	0.136s	1	0.705	0.827	0.539	0.771	0.693	12

1.4.4. DBSCAN $\epsilon = 0.7$

En la presente sección se entreno al algoritmo `DBSCAN_2` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_2, "DBSCAN_2", data, labels, False)`.
Los resultados obtenidos se presentan en 5.

Tabla 5: Resultados de `DBSCAN_2` sin muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_2	0.011s	1	0.536	0.692	0.263	0.630	0.377	43

1.4.5. DBSCAN $\epsilon = 0.2$

En la presente sección se entreno al algoritmo `DBSCAN_3` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_3, "DBSCAN_3", data, labels, False)`.
Los resultados obtenidos se presentan en 6.

Tabla 6: Resultados de `DBSCAN_3` sin muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_3	0.014s	NaN	NaN	NaN	NaN	NaN	NaN	0

1.4.6. DBSCAN con ruido

En la presente sección se entreno al algoritmo `DBSCAN_1` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_1, "DBSCAN_4", data, labels, True)`.
Los resultados obtenidos se presentan en 7.

Tabla 7: Resultados de `DBSCAN_1` con muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_4	0.016s	0	0.365	0.165	0.002	0.113	−0.332	13

1.4.7. DBSCAN $\epsilon = 0.7$ con ruido

En la presente sección se entreno al algoritmo `DBSCAN_2` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_2, "DBSCAN_5", data, labels, True)`.
Los resultados obtenidos se presentan en 8.

Tabla 8: Resultados de `DBSCAN_2` con muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_5	0.009s	1	0.425	0.490	0.072	0.422	−0.009	44

1.4.8. DBSCAN $\epsilon = 0.2$ con ruido

En la presente sección se entreno al algoritmo `DBSCAN_3` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(DBSCAN_3, "DBSCAN_6", data, labels, True)`.
Los resultados obtenidos se presentan en 9.

Tabla 9: Resultados de `DBSCAN_3` con muestras ruido.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
DBSCAN_6	0.012s	NaN	NaN	NaN	NaN	NaN	NaN	1

1.4.9. Hierarchical Agglomerative

En la presente sección se entreno al algoritmo `Agglomerative_1` mediante el uso de la función implementada en 1.3, por medio de `bench_clustering2(Agglomerative_1, "Agglomerative_1 ↪ ", data, labels, False)`.
Los resultados obtenidos se presentan en 10.

Tabla 10: Resultados de `Agglomerative_1`.

init	time	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
Agglomerative_1	0.020s	0	0.356	0.354	0.168	0.341	0.140	8

1.4.10. Repetición con PCA

En la presente sección se repitió el entrenamiento de los algoritmos previos, pero con el *PCA* mencionado en 1.4.
El código implementado se presenta en 7.

Código 7: Entrenamiento con *PCA*.

```
bench_k_means(KMeans_1_PCA, "KMeans_1_PCA", reduced_data, labels)
bench_k_means(Kmeans_2_PCA, "Kmeans_2_PCA", reduced_data, labels)
bench_clustering2(DBSCAN_1_PCA, "DBSCAN_1_PCA", reduced_data, labels, False)
bench_clustering2(DBSCAN_2_PCA, "DBSCAN_2_PCA", reduced_data, labels, False)
bench_clustering2(DBSCAN_3_PCA, "DBSCAN_3_PCA", reduced_data, labels, False)
bench_clustering2(DBSCAN_1_PCA, "DBSCAN_4_PCA", reduced_data, labels, True)
bench_clustering2(DBSCAN_2_PCA, "DBSCAN_5_PCA", reduced_data, labels, True)
bench_clustering2(DBSCAN_3_PCA, "DBSCAN_6_PCA", reduced_data, labels, True)
bench_clustering2(Agglomerative_1_PCA, "Agglomerative_1", reduced_data, labels, False)
```

Los resultados obtenidos se presentan en 11.

Tabla 11: Resultados de `KMeans_1`.

init	time	inertia	homo	compl	v-means	ARI	AMI	silhoutte	n_clusters
KMeans_1_PCA	0.037s	346	0.206	0.210	0.208	0.094	0.192	0.369	8
KMeans_2_PCA	0.038s	346	0.203	0.207	0.205	0.092	0.189	0.384	8
DBSCAN_1_PCA	0.005s	N/A	NaN	NaN	NaN	NaN	NaN	NaN	1
DBSCAN_2_PCA	0.006s	N/A	NaN	NaN	NaN	NaN	NaN	NaN	1
DBSCAN_3_PCA	0.006s	N/A	0	0.283	0.284	0.068	0.212	0.088	20
DBSCAN_4_PCA	0.005s	N/A	0	0.116	0.011	0.000	0.005	0.386	2
DBSCAN_5_PCA	0.006s	N/A	0	0.335	0.010	0.000	0.005	0.435	2
DBSCAN_6_PCA	0.005s	N/A	0	0.202	0.198	0.037	0.143	−0.225	21
Agglomerative_1_PCA	0.009s	N/A	0	0.226	0.223	0.102	0.206	0.345	8

1.5. Análisis de los resultados

En la presente sección se analizaran los resultados expuestos en las secciones de 1.4.

1.5.1. Comparación de algoritmos

En la presente sección se compararan los algoritmos implementados en base a cuatro métricas:

1. *Completeness*.
2. *Homogeneity*.
3. *v-measure*.
4. *Silhoutte*.

Se indicara el mejor algoritmo según cada métrica y el peor algoritmo según cada métrica. Los resultados se exponen a continuación.

Tabla 12: Algoritmos con las mejores métricas.

Métrica	Algoritmo	Valor
Completeness	`DBSCAN_1`	0.705
Homogeneity	`DBSCAN_1` `DBSCAN_2`	1
v-measure	`DBSCAN_1`	0.827
Silhoutte	`DBSCAN_1`	0.693

Tabla 13: Algoritmos con las peores métricas.

Métrica	Algoritmo	Valor
Completeness	`DBSCAN_4_PCA`	0.116

Homogeneity	`DBSCAN_1` con ruido
	`DBSCAN_3_PCA`
	`DBSCAN_1_PCA` con ruido
	`DBSCAN_2_PCA` con ruido
	`DBSCAN3_PCA` con ruido
	`Agglomerative_1_PCA`
		0

v-measure	`DBSCAN_2_PCA` con ruido	0.010
Silhoutte	`DBSCAN_1` con ruido	-0.332

A partir de 12 es claro notar que el mejor algoritmo implementado es `DBSCAN_1`. Por otra parte, a partir de 13 no es claro notar el peor algoritmo implementado, pero si es claro que tanto el *PCA* implementado como el considerar las muestras ruido empeora el rendimiento de los algoritmos.

1.5.2. Algoritmo DBSCAN

En la presente sección se analizaran los algoritmos DBSCAN implementados.
Por ultimo, es relevante mencionar que el mejor algoritmo mencionado previamente no es perfecto, lo cual es claro al considerar su numero de clusters.

1.5.2.1. Casos sin métricas

Los casos en que no fueron posible calcular las métricas de los algoritmos DBSCAN implementados se presentan en 14

Tabla 14: Casos sin métricas

Algoritmo	Numero de clusters	Muestras ruido
`DBSCAN_3`	0	No
`DBSCAN_3`	1	Si
`DBSCAN_1_PCA`	1	No
`DBSCAN_2_PCA`	1	No

Es claro notar que el algoritmo `DBSCAN_3`, el cual corresponde al caso en que $\epsilon = 0.2$, no logra realizar clusters sobre el conjunto original, pero si para el conjunto reducido, aunque sin obtener buenos resultados, pues en esos casos genera una numero de clusters ≥ 20, lo cual es una cantidad muy superior a la cantidad de clases (8).
Por otra parte, se logra apreciar que al momento de utilizar *PCA* y no considerar las muestras ruido, los algoritmos `DBSCAN_1_PCA` y `DBSCAN_2_PCA` no logran realizar el clustering de manera apropiada, y al momento de considerar las muestras ruido, el numero de clusters es 2, un valor muy inferior a la cantidad de clases.

1.5.2.2. Numero de clusters

El numero de clusters de cada algoritmo se presentan en 15.

Tabla 15: Numero de clusters de los algoritmos DBSCAN.

Algoritmo	Numero de clusters	Muestras ruido
DBSCAN_1	12	No
DBSCAN_2	43	No
DBSCAN_3	0	No
DBSCAN_1	13	Si
DBSCAN_2	44	Si
DBSCAN_3	1	Si
DBSCAN_1_PCA	1	No
DBSCAN_2_PCA	1	No
DBSCAN_3_PCA	20	No
DBSCAN_1_PCA	2	Si
DBSCAN_2_PCA	2	Si
DBSCAN_3_PCA	21	Si

Si bien existen un claro mejor algoritmo según lo expuesto anteriormente, su numero de clusters no es la cantidad de clases.
El resto de los algoritmos se alejan aún más de la cantidad de clases y sus métricas son, en general, inferiores a las del mejor algoritmo. Lo cual ocurre debido que ambas cosas tienen estrecha relación, pues las métricas indican la calidad de los clusters realizados por el algoritmo.

1.5.2.3. Efecto de considerar las muestras ruido

Al incluir las muestras ruido en el calculo de las métricas:

DBSCAN Todos los valores de estas disminuyen de forma sustancial.

DBSCAN $\epsilon = 0.7$ Todos los valores menos *Homogeneity* disminuyen de forma sustancial.

DBSCAN $\epsilon = 0.2$ No se modifican (ambos son NaN).

DBSCAN PCA Se generan valores, pues previamente eran NaN.

DBSCAN $\epsilon = 0.7$ **PCA** Se generan valores, pues previamente eran NaN.

DBSCAN $\epsilon = 0.2$ **PCA** Todos disminuyen.

1.5.3. Efecto de utilizar PCA

El efecto que tiene el utilizar PCA sobre las métricas es que estas, de manera general, disminuyen significativamente. La única métrica que aumenta en la mayoría de las ocasiones es *Silhouette*.
Además, para DBSCAN_1 y DBSCAN_2 sin ruido, no realiza clustering.
Por otra parte, en el caso de DBSCAN_3 con y sin ruido, logra realizar clustering.

2. Conclusiones

2.1. Aprendizajes

A continuación se describen mis aprendizajes.

Algoritmos de clustering He aprendido a implementar distintos algoritmos de clustering en python. He aprendido que no es necesario visualizar el resultado del clustering para definir si este es bueno o no. He aprendido que los tiempos de entrenamiento de los algoritmos básicos de clustering son bastante pequeños, menos de un segundo en el caso particular de este problema.

Métricas de clusters He aprendido acerca de las diversas métricas aplicadas a un algoritmo de clustering, de sus significados y relevancia a la hora de evaluar el clustering realizado por el algoritmo. He aprendido que el numero de clusters no es la métrica que entrega la información más relevante del clustering realizado. He aprendido, según lo realizado en la sección de análisis cuales son las métricas más relevantes y cual es el efecto sobre las métricas de realizar un PCA (quizas incorrecto) sobre el conjunto de datos.

2.2. Dificultades

- Desconocimiento acerca de las métricas de un clustering.
- Desconocimiento de las funciones de *scikit-learn*.
- Desconocimiento de como realizar un PCA en python.
- Implementación de la inclusión (o no) de las muestras ruido.

2.3. Evaluación de resultados

Los resultados son acordes a la teoría, pues debido a que los datos son biológicos y en dichos casos suelen existir varios outliers, el mejor algoritmo sera el que tenga la mejor capacidad para manejar outliers, en este caso DBSCAN. Sin embargo, el valor de ϵ es de suma importancia.
Hierarchical Agglomerative debiese ser el que le sigue, pues posee la capcidad de capturar estructuras complejas.
K-Means es útil en este caso pues sabemos de antemano la cantidad de clases, si fuese desconocida los resultados para dicho algoritmos se verían, en teoría, perjudicados.
Además, en el caso de utilizar un PCA de baja dimensionalidad comparado al conjunto original, se generan clusters menos definidos, pues el PCA no contiene las componentes más relevantes.

Anexo A. Código

Código A.1: Código.

```
# -*- coding: utf-8 -*-
"""Tarea 5 – Clustering EL4106 - Diego Vega.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1VwrQfdY_hXpOVeZ3XG6yiG5PBszB6tcZ
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
from time import time
from sklearn import metrics
from sklearn.pipeline import make_pipeline
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA

"""# 1)

"""

DataSet = 'Data_Cortex_Nuclear.xls'
DataFrame = pd.read_excel(DataSet)
DataFrame = DataFrame.drop('BCL2_N', axis=1)
DataFrame = DataFrame.dropna()
print(DataFrame)

"""# 2)"""

def bench_k_means(kmeans, name, data, labels):
    """Benchmark to evaluate the KMeans initialization methods.

    Parameters
    ----------
    kmeans : KMeans instance
        A :class:`~sklearn.cluster.KMeans` instance with the initialization
        already set.
    name : str
        Name given to the strategy. It will be used to show the results in a
```

```
43          table.
44      data : ndarray of shape (n_samples, n_features)
45          The data to cluster.
46      labels : ndarray of shape (n_samples,)
47          The labels used to compute the clustering metrics which requires some
48          supervision.
49      """
50      t0 = time()
51      estimator = make_pipeline(kmeans).fit(data)
52      fit_time = time() - t0
53      results = [name, fit_time, estimator[-1].inertia_]
54
55      # Define the metrics which require only the true labels and estimator
56      # labels
57      clustering_metrics = [
58          metrics.homogeneity_score,
59          metrics.completeness_score,
60          metrics.v_measure_score,
61          metrics.adjusted_rand_score,
62          metrics.adjusted_mutual_info_score,
63      ]
64      results += [m(labels, estimator[-1].labels_) for m in clustering_metrics]
65
66      # The silhouette score requires the full dataset
67      results += [
68          metrics.silhouette_score(
69              data,
70              estimator[-1].labels_,
71              metric="euclidean",
72              sample_size=300,
73          )
74      ]
75
76      results.append(kmeans.n_clusters)
77      header = ["init", "time", "inertia", "homo", "compl", "v-meas", "ARI", "AMI", "silhouette",
        ↪ "n_clusters"]
78      formatter_header = (
79          "{:8s}\t{:4s}\t{:7s}\t{:4s}\t{:5s}\t{:6s}\t{:3s}\t{:3s}\t{:10s}\t{:4s}"
80      )
81      print(formatter_header.format(*header))
82      # Show the results
83      formatter_result = (
84          "{:8s}\t{:.3f}s\t{:.0f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:9.0f}"
85      )
86      print(formatter_result.format(*results))
87
88  """# 3)"""
```

```
89
90  def bench_clustering2(method, name, data, labels, calculate_all):
91      """Benchmark to evaluate the DBSCAN and Aglomerative initialization methods.
92
93      Parameters
94      ----------
95      methods : Clustering instance
96          A :class: `~sklearn.cluster.DBSCAN` or `sklearn.cluster.AgglomerativeClustering`
     ↪ instance with the initialization
97          already set.
98      name : str
99          Name given to the strategy. It will be used to show the results in a
100         table.
101     data : ndarray of shape (n_samples, n_features)
102         The data to cluster.
103     labels : ndarray of shape (n_samples,)
104         The labels used to compute the clustering metrics which requires some
105         supervision.
106     """
107     t0 = time()
108     estimator = make_pipeline(method).fit(data)
109     fit_time = time() - t0
110     results = [name, fit_time]
111
112     # Define the metrics which require only the true labels and estimator
113     # labels
114     clustering_metrics = [
115         metrics.homogeneity_score,
116         metrics.completeness_score,
117         metrics.v_measure_score,
118         metrics.adjusted_rand_score,
119         metrics.adjusted_mutual_info_score,
120     ]
121
122     if isinstance(method, sklearn.cluster.DBSCAN):
123       n_clusters = len(set(estimator[-1].labels_)) - (1 if -1 in estimator[-1].labels_ else 0)
124       if calculate_all and -1 in estimator[-1].labels_:
125         predicted_labels = np.where(estimator[-1].labels_ == -1, n_clusters, estimator[-1].
     ↪ labels_)
126         n_clusters += 1
127     elif isinstance(method, sklearn.cluster.AgglomerativeClustering):
128       n_clusters = method.n_clusters
129
130     if n_clusters > 1:
131       if not calculate_all and isinstance(method, sklearn.cluster.DBSCAN):
132         mask = estimator[-1].labels_ != -1
133       else:
```

```
      mask = np.ones(len(estimator[-1].labels_), dtype=bool)

    results += [m(labels[mask], estimator[-1].labels_[mask]) for m in clustering_metrics]

    # The silhouette score requires the full dataset
    results += [
        metrics.silhouette_score(
            data[mask],
            estimator[-1].labels_[mask],
            metric="euclidean",
            sample_size=300,
        )
    ]

  else:
    print("One or no clusters found")
    results += [np.nan] * 6

  results.append(n_clusters)
  header = ["init", "time", "homo", "compl", "v-meas", "ARI", "AMI", "silhouette", "
   ↪ n_clusters"]
  formatter_header = (
      "{:8s}\t{:4s}\t{:4s}\t{:5s}\t{:6s}\t{:3s}\t{:3s}\t{:10s}\t{:4s}"
  )
  print(formatter_header.format(*header))
  # Show the results
  formatter_result = (
      "{:8s}\t{:.3f}s\t{:.0f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:.3f}\t{:10.0f}"
  )
  print(formatter_result.format(*results))

"""# 4)


"""

data = DataFrame.drop('class', axis=1).select_dtypes(include=['float64', 'int64']).values
labels = DataFrame['class'].values
KMeans_1 = KMeans(init='random', n_clusters=8, n_init=10)
Kmeans_2 = KMeans(init='k-means++', n_clusters=8 , n_init=10)
DBSCAN_1 = DBSCAN()
DBSCAN_2 = DBSCAN(eps=0.7)
DBSCAN_3 = DBSCAN(eps=0.2)
Agglomerative_1 = AgglomerativeClustering(n_clusters=8)
reduced_data = PCA(n_components=2).fit_transform(data)
KMeans_1_PCA = KMeans_1.fit(reduced_data)
Kmeans_2_PCA = Kmeans_2.fit(reduced_data)
```

```
DBSCAN_1_PCA = DBSCAN_1.fit(reduced_data)
DBSCAN_2_PCA = DBSCAN_2.fit(reduced_data)
DBSCAN_3_PCA = DBSCAN_3.fit(reduced_data)
Agglomerative_1_PCA = Agglomerative_1.fit(reduced_data)

"""## a)"""

bench_k_means(KMeans_1, "KMeans_1", data, labels)

"""## b)

"""

bench_k_means(Kmeans_2, "Kmeans_2", data, labels)

"""## c)"""

bench_clustering2(DBSCAN_1, "DBSCAN_1", data, labels, False)

"""## d)

"""

bench_clustering2(DBSCAN_2, "DBSCAN_2", data, labels, False)

"""## e)"""

bench_clustering2(DBSCAN_3, "DBSCAN_3", data, labels, False)

"""## f)"""

bench_clustering2(DBSCAN_1, "DBSCAN_4", data, labels, True)

"""## g)"""

bench_clustering2(DBSCAN_2, "DBSCAN_5", data, labels, True)

"""## h)"""

bench_clustering2(DBSCAN_3, "DBSCAN_6", data, labels, True)

"""## i)"""

bench_clustering2(Agglomerative_1, "Agglomerative_1", data, labels, False)

"""## j)"""

```

```
227 bench_k_means(KMeans_1_PCA, "KMeans_1_PCA", reduced_data, labels)
228 bench_k_means(Kmeans_2_PCA, "Kmeans_2_PCA", reduced_data, labels)
229 bench_clustering2(DBSCAN_1_PCA, "DBSCAN_1_PCA", reduced_data, labels, False)
230 bench_clustering2(DBSCAN_2_PCA, "DBSCAN_2_PCA", reduced_data, labels, False)
231 bench_clustering2(DBSCAN_3_PCA, "DBSCAN_3_PCA", reduced_data, labels, False)
232 bench_clustering2(DBSCAN_1_PCA, "DBSCAN_4_PCA", reduced_data, labels, True)
233 bench_clustering2(DBSCAN_2_PCA, "DBSCAN_5_PCA", reduced_data, labels, True)
234 bench_clustering2(DBSCAN_3_PCA, "DBSCAN_6_PCA", reduced_data, labels, True)
235 bench_clustering2(Agglomerative_1_PCA, "Agglomerative_1", reduced_data, labels, False)
```